

Convolutions, batch normalisation and dropout on Fashion MNIST

Vincent Brand | 1447806 | Machine Learning – MADS, HAN

2026 | <https://github.com/vincentbrand/MADS-ML-Vincent>

1 Hypothesis

Ch1's dense network plateaued near 0.85 — extra width helped little, extra depth not at all — which motivates convolutions and regularisation. I test three hypotheses. **(H1)** A small CNN should reach the dense baseline's accuracy at *far fewer* parameters, since convolution's parameter sharing and local connectivity exploit the 2-D structure that flattening the 784 pixels destroys (Ch10). **(H2)** BatchNorm should barely help a shallow CNN but stabilise a deeper one and make it learning-rate tolerant, by smoothing a loss landscape whose roughness only binds as depth grows (Ch11). **(H3)** Dropout should shrink the train-validation gap and help accuracy *only* when the model overfits; on an under-capacity model it should *hurt* by removing usable capacity (Ch9).

2 Experiment design

The CNN stacks conv blocks in an `nn.ModuleList` — `Conv2d(3 × 3)→ReLU→optional BatchNorm2d→MaxPool2d`, depth set by `num_conv_layers` — then an `AvgPool2d` and a dense head with optional `Dropout`; the dense baseline is the Ch1 model. All runs share `CrossEntropyLoss`, Adam, `ReduceLROnPlateau` and one fixed step budget for fair comparison, logged to MLflow. H1 pits dense (64×32 to 512×256) against CNNs (2–3 layers, 16–32 filters) by parameter count (Fig. 1); H2 is a depth×BatchNorm×lr grid (Fig. 2); H3 varies dropout on a small (64×32) and a large (512×512) model (Fig. 3).

3 Results

H1 only half held (Fig. 1): the CNNs are far more parameter-efficient — a 6k-param CNN already learns and the 24k-param 3-layer CNN reaches 0.778 at a 0.6% gap — yet none beat the dense nets on raw accuracy (best dense 0.845 at 536k params). The CNN curve still climbs steeply (the third conv layer alone gave $0.678 \rightarrow 0.778$), so the binding limit is the narrow 32-filter extractor, not the principle. **H2 confirmed** (Fig. 2): BatchNorm helped in every cell, most where optimisation struggled (shallow, $\text{lr} = 10^{-3}$: $0.684 \rightarrow 0.838$); without it the net needed the higher lr, with it $\text{lr} = 10^{-3}$ gave the best model (deep+BN, 0.889) — BatchNorm *decoupled* accuracy from the learning rate. **H3 confirmed by regime** (Fig. 3): dropout 0.5 shrank the large model's gap ($0.0159 \rightarrow 0.0133$) but on the small model left the gap flat and cut accuracy ($0.837 \rightarrow 0.783$).

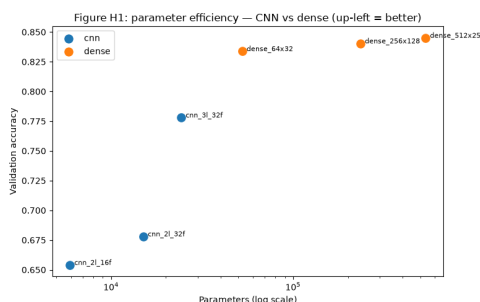


Figure 1: H1 — validation accuracy vs. parameter count (log; up-and-left is better). CNNs use far fewer parameters but do not reach the dense nets' accuracy.

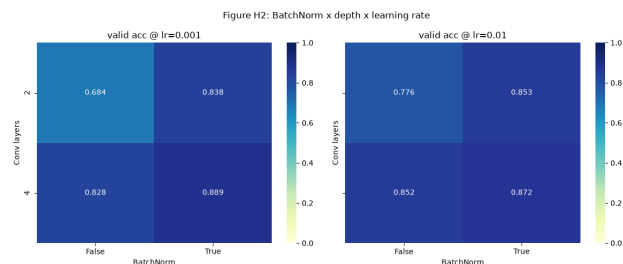


Figure 2: H2 — valid. accuracy over conv depth × BatchNorm at two lr. BatchNorm helps everywhere and is nearly lr-independent.

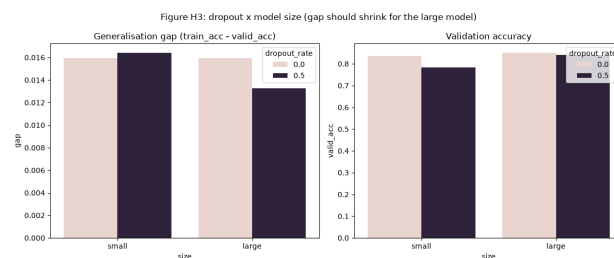


Figure 3: H3 — generalisation gap (left) and valid. accuracy (right), dropout 0.0 vs. 0.5. The gap shrinks only on the large model.

4 Conclusions

BatchNorm was the most valuable addition: it raised accuracy and made the learning rate far less critical (the smoother-landscape effect of Ch11), with a payoff that grew with depth. Convolutions gave efficiency and tighter generalisation but, at the filter widths I could afford, did not overtake the dense baseline — an honest mismatch with H1 that points at the extractor, not the inductive bias. Dropout helped only the overparameterised model, and only modestly, since Fashion-MNIST barely overfits here. Next I would widen the CNN, drop the global-average head, add augmentation, and test on data where regularisation bites harder.